Figure 10.12. Velocity for this team is inconsistent, but total completed tasks are actually increasing. This is a typical indicator that the team completes tasks consistently despite changing commitments and/or requirements.

Figure 10.13. Where to find data to measure effective requirements

Figure 10.14. The team's dashboard with labels, estimate distribution, estimate health, and lead time. These metrics helped the team stay aware of how well they were estimating and delivering.

Figure 10.15. The next set of charts used by the development team included good versus bad builds, unit test coverage, and mutant coverage.

Figure 10.16. The dashboard tracking recidivism and bugs versus total tasks to help the team track how well they're working together across disciplines.

Appendix A. DIY analytics using ELK

Figure A.1. The high-level component diagram of the analytics system

Figure A.2. A closer look at the flow of data through the analytics system

Figure A.3. How Vagrant, Puppet, and VirtualBox interact with your development environment

Figure A.4. A few simple ways to create new graphs in Kibana

Appendix B. Collecting data from source systems with Grails

Figure B.1. The focus of this appendix, the Grails portion of measurementor

Figure B.2. A closer look into the Grails/Mongo-based data collection system

Figure B.3. Where to find the three main components of the app: domain objects, services, and jobs

Figure B.4. Representing the response as a grouping of domain objects